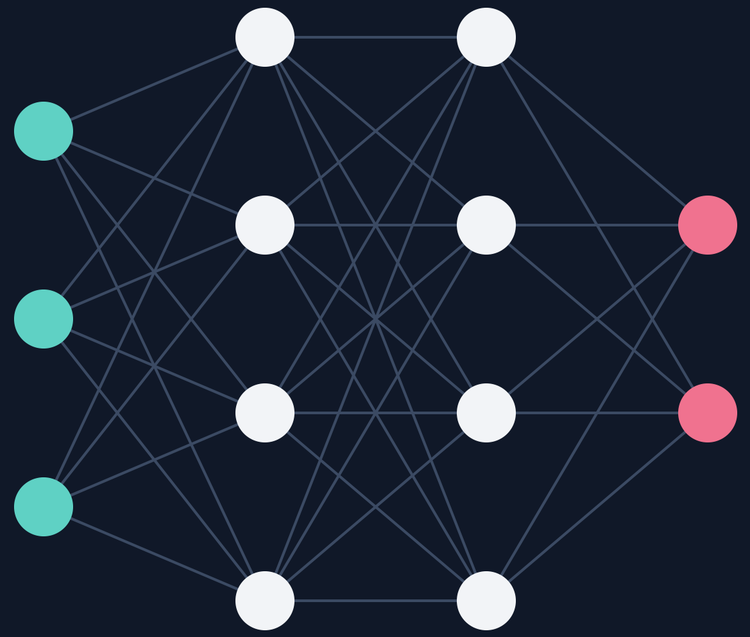
AIML-501 · WORKSHOP 2 · ARTIFACT 02



Neural Network Components

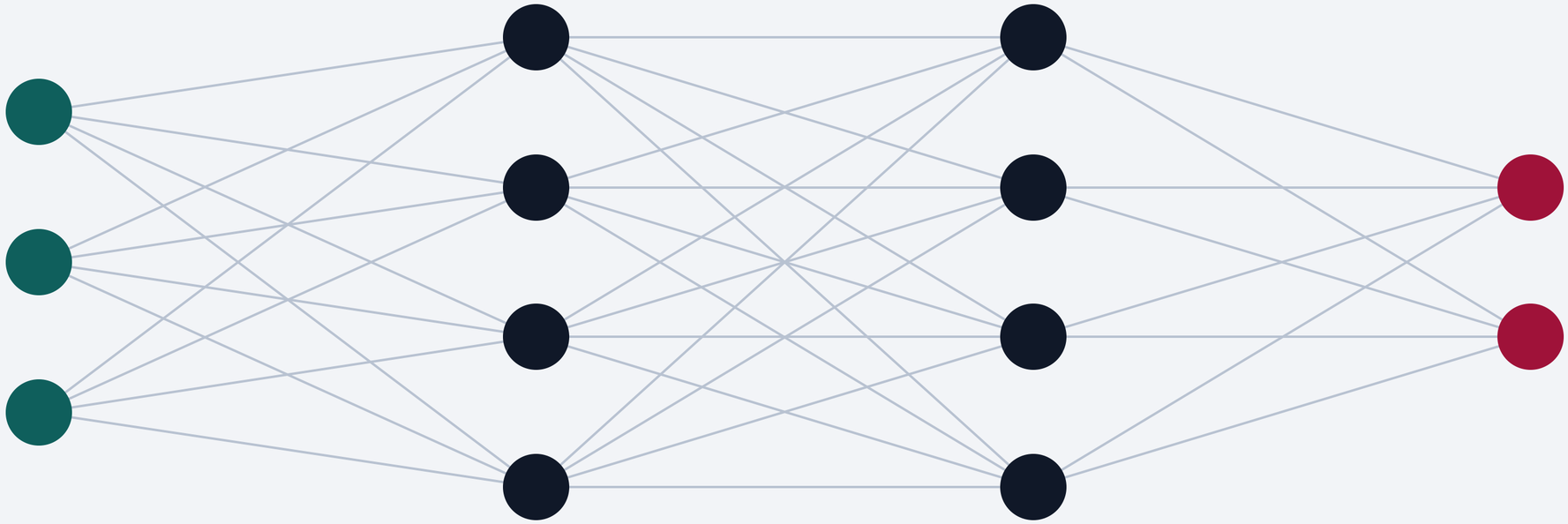
Layers, neurons, weights, activation functions, loss functions and optimization algorithms, explained visually.

Shubhra Shambhavi · Model Development · IWU

THE BIG PICTURE

Overall architecture

Data flows left to right. Every line is a connection that carries a weight.



Input layer
Receives the data

Hidden layer 1
Learns simple patterns

Hidden layer 2
Builds complex patterns

Output layer
Produces the prediction

HOW I ORGANIZED IT

Grouping the six components

I sorted them by the question each one answers about the network.

Structure

What is it made of?

Layers and neurons. They set the shape and the capacity of the network before any learning starts.

Computation

How does a neuron work?

Weights and activation functions. They turn a neuron's inputs into its output.

Learning

How does it improve?

Loss functions and optimization algorithms. They score the errors and then adjust the weights.

Weights sit in the middle on purpose. They are used in computation and changed by learning.

The next six slides follow this order.

COMPONENT 1 OF 6

Layers

A layer is a group of neurons that process data together, then pass the result to the next layer.

Input layer

Receives the raw features. It does no learning itself. It has one node per feature, such as X1 and X2 in the Playground.

Hidden layers

Do the learning. Each added layer builds more complex features from the output of the layer before it.

Output layer

Produces the final prediction, such as a class label or a number. Its size matches the task.

More layers add depth. More neurons per layer add width.

COMPONENT 2 OF 6

Neurons

A neuron is the basic unit of a network. It takes inputs, weighs them, adds a bias and applies an activation function.

Inputs

Values from the data or the previous layer.

Weighted sum

Each input is multiplied by its weight, then added with a bias.

Activation

A function bends the sum so the neuron can learn curves.

Output

The result goes to every neuron in the next layer.

$$a = f(w1 \cdot x1 + w2 \cdot x2 + \dots + b)$$

Neurons in the same layer see the same inputs but learn different features.

COMPONENT 3 OF 6

Weights and biases

A weight sets how strongly one input influences a neuron. A bias shifts the neuron's result up or down.

Weight	Effect on the neuron
Large positive	The input strongly raises the output
Close to zero	The input is mostly ignored
Negative	The input lowers the output

In the Playground, the thickness of each connection line shows how large its weight is.

Learned, not set

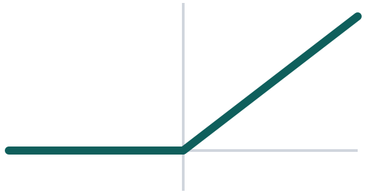
Weights and biases start as small random numbers. Training adjusts them again and again. They are what the network learns.

A bias lets a neuron respond even when every input is zero.

COMPONENT 4 OF 6

Activation functions

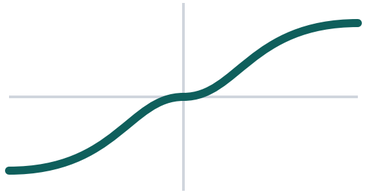
An activation function turns a neuron's sum into its output. Without it, stacked layers collapse into one straight line.



ReLU

Output 0 and up

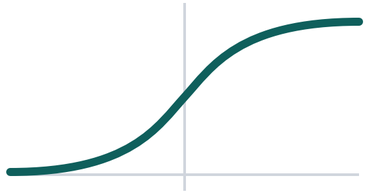
A common default for hidden layers.



Tanh

Output −1 to 1

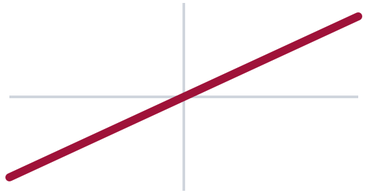
Centers values around zero.



Sigmoid

Output 0 to 1

Suits yes or no outputs.



Linear

Output any value

Adds no curve, so it fits only straight boundaries.

The Playground lets you switch between all four.

COMPONENT 5 OF 6

Loss functions

A loss function scores how wrong the predictions are. Training tries to make that score small.

Mean squared error

Averages the squared gap between each prediction and the true value. Used when predicting numbers.

Cross entropy

Punishes confident wrong class predictions most. Used for classification.

Training vs test loss

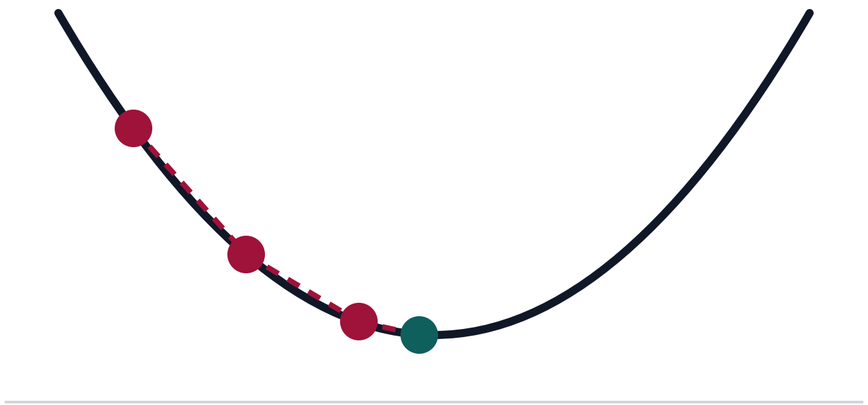
Training loss is the error on data the network learns from. Test loss is the error on unseen data. A widening gap signals overfitting.

The Playground shows both losses while the network trains.

COMPONENT 6 OF 6

Optimization algorithms

An optimizer updates the weights to reduce the loss. Gradient descent steps downhill on the loss surface.



Each step moves the weights downhill.

Gradient descent

Measures how the loss changes with each weight, then nudges every weight downhill.

Mini batches

Uses a small batch of examples for each update, which is faster and adds helpful noise.

Learning rate

Sets the step size. Too small trains slowly. Too large overshoots and may never settle.

Learning rate and batch size are both controls in the Playground.

PUTTING IT TOGETHER

How a network trains

Training repeats a four step cycle until the loss stops improving.

1

Forward pass

Inputs flow through every layer to make a prediction.

Weights · Activation

2

Compute loss

The prediction is compared with the true label.

Loss function

3

Backward pass

The error is traced back to find each weight's share of it.

Backpropagation

4

Update weights

The optimizer nudges each weight to reduce the loss.

Optimizer

Then repeat. One full pass through the training data is one epoch.

Each tag shows which component does the work in that step.

HANDS-ON PRACTICE

Playground experiments

Four settings I varied in the TensorFlow Neural Network Playground, and what to watch for with each one.

Data type

Circle, XOR, Gaussian, Spiral

Each dataset needs a different boundary shape, so each tests the network differently.

Noise

Low to high

Noise blurs the classes. Watch the gap between training and test loss.

Features

X1, X2, squares, product, sine

Extra features hand the network curved building blocks to work with.

Layers and neurons

Fewer to more

More layers and neurons add capacity and complexity, and can slow training.

For every change I compared training time, model complexity and performance.

Source: TensorFlow Neural Network Playground, playground.tensorflow.org

SUMMARY

Why visualizing networks matters

See inside the black box

Watching weights and decision boundaries change shows what the network is learning.

Spot problems early

A flat loss curve or a widening gap between training and test loss shows a fault.

Build intuition

Changing one setting at a time shows how depth, width and activations trade off.

Key insights

- Layers and neurons set how much the network can learn.
- Weights are what the network learns. Activations make that learning nonlinear.
- The loss measures the error and the optimizer reduces it.
- Capacity must match the data, or the network overfits.